Buffer-Bug Dataset Correction

`sum_buffers()` in `scripts/collection/collect_query.py` recursively summed PostgreSQL's per-node `Shared Hit/Read Blocks` up the plan tree. PostgreSQL reports these fields cumulatively — the root node already holds the whole-query total — so summing multi-counted every buffer touch once per ancestor level. This is the same bug fixed in the live gate (`cetp_gate.py`'s `root_buffers()`); `tpch_dataset.csv` was collected before the fix and carries the inflated values. Raw EXPLAIN plans from the original 5-EC2-machine collection were not retained, so per-row correction from saved data is impossible.

Step 1: local regeneration

Buffer counts for a query against fixed data are a property of the query plan and the data, not of the machine — the same query against the same TPC-H SF1 data yields the same page-touch footprint on any host. This was verified against a local PostgreSQL 15.17 SF1 instance already set up for a prior investigation (Addition 4b): exact row-count match to the collection guide (6,001,215 `lineitem` rows), same 21 patched query files (`scripts/collection/collect_query.py` and `scripts/collection/check_parallelism.py` share `QUERY_DIR`), same index set, and all planner-relevant settings (`random_page_cost=4`, `seq_page_cost=1`, `effective_cache_size=4GB`, `shared_buffers=128MB`, `max_parallel_workers_per_gather=2`) at PostgreSQL factory defaults — matching the prior finding that the original collection guide never mentions custom tuning. Residual uncertainty: the local planner's `cost` estimate differs from the stored dataset's by under 1% (different ANALYZE/vacuum history), and the hit/read *split* (not the total) reflects whichever buffer-cache warmth state the local instance was in at capture time. `cost` was never part of this bug and is untouched regardless.

`EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` was run for all 21 queries; root-node values were extracted directly (`root_buffers()`) alongside the old buggy tree-sum for comparison. Full table: `results/buffer_bug_correction_table.csv`.

query	correct hit	correct read	buggy hit	buggy read	hit inflation	read inflation
q1	14,799	95,585	73,963	477,925	5.00x	5.00x
q6	2,220	108,148	8,880	432,592	4.00x	4.00x
q14	163	114,188	652	574,758	4.00x	5.03x
q11	42,749	20,040	197,986	84,932	4.63x	4.24x
q17	226,722	110,705	923,799	338,911	4.07x	3.06x
q10	176,780	85,991	1,944,203	938,621	11.00x	10.92x
q18	302,455	149,372	3,629,164	1,731,726	12.00x	11.59x

Inflation varies widely, from 3.06x to 12.00x across the 21-query workload — confirmed, not a constant multiplier. It tracks plan node count (more joins/scans in the tree → more double-counting), so it cannot be corrected by a single global scale factor; it had to be measured per query.

Step 2: corrected dataset

`tpch_dataset_corrected.csv` was written by substituting each row's `shared_hit` / `shared_read` with the correct, machine-invariant value for that `query_id` from Step 1. `cost`, `time_ms`, `rows`, and `machine_id` are byte-identical to the original — verified programmatically. `total_buffers` and `io_ratio` are not stored columns (both scripts derive them from `shared_hit + shared_read` on load); they come out corrected automatically downstream. `tpch_dataset.csv` was not modified (md5 `3452508f6334ac10f2b552ec92020634`, mtime unchanged).

Step 3: bottleneck classifier — materially worse, and the mechanism is fully explained

Re-running Phase 5's exact leave-one-query-out RandomForest (`n_estimators=300`, `class_weight="balanced"`, `random_state=42`, unchanged):

	macro F1	compute F1	bandwidth F1	io F1	mixed F1
original (buggy)	0.562	0.667	0.571	0.250	0.762
corrected	**0.375**	0.500	0.600	**0.000**	0.400

45 of 105 (query, machine) rows flip predicted label — always in whole groups of 5 (9 queries × 5 machines), because correcting the bug makes every feature machine-invariant per query. `cost` and `rows` were *already* identical across all 5 machines for every query in the original data (verified: zero queries had cost or rows variation by machine); `shared_hit` / `shared_read` were the *only* source of cross-machine spread the classifier had within a query. Once corrected to their true hardware-independent values, that spread goes to exactly zero: the training set collapses from 105 distinct feature vectors to 21 unique ones (each repeated 5×), and leave-one-query-out generalization measurably suffers as a result. This is not a modeling artifact — it is the direct, mechanical consequence of buffers actually being hardware-independent, which the buggy data's incidental per-machine noise had been (unintentionally) padding out. **The io class, already the weakest link in the original report, now has zero recall** — every io-labeled row predicts as `mixed`. The classifier's real accuracy is materially lower than reported, and the reported 0.562 was, in part, inflated by the bug itself.

No correlation or feature-importance computation in this project uses `total_buffers` or `io_ratio` as the correlated/ranked variable directly (checked `phase7_final_evaluation.py`, `addition4_reevaluate.py`, `addition4b_mechanism_check.py`) — every `.corr()` call uses `compute_ratio` or `bandwidth_ratio` against `scaling_factor_p50`, and query subsetting uses the hand-assigned ground-truth `BOTTLENECK_LABELS`, never the classifier's predictions. `feature_importances_` is never called anywhere. These tables are unaffected, unchanged.

Step 4: the scaling regressor uses buffer features directly — full corrected re-run

`ALLOWED_FEATURES` for the leave-one-machine-out GBR quantile model includes `dev_shared_hit`, `dev_shared_read`, `dev_total_buffers`, `dev_io_ratio` directly, plus `p_compute` / `p_bandwidth` / `p_io` / `p_mixed` from the (now-corrected) bottleneck classifier and the `analytical_scaling` formula built from those probabilities. This required the full re-run, not just a sensitivity check.

Phase 7 headline (OLD single-core compute), buggy vs. corrected:

approach	buggy MAPE	corrected MAPE	Δ
naive (=1.0)	31.97%	31.97%	0.00
naive-linear (1/bandwidth_ratio)	23.03%	23.03%	0.00
analytical roofline	31.14%	31.12%	−0.02pp
bottleneck-gated	28.06%	28.06%	0.00
learned quantile (p50)	31.26%	32.79%	+1.53pp

Significance (cluster bootstrap, `N=2000`, same seed): naive-linear still significantly beats analytical (CI [−11.80, −4.38], was [−11.82, −4.40]) and gated (CI [−7.17, −3.02], identical); still statistically indistinguishable from the learned model (CI [−22.78, 0.33], was [−22.38, 2.41]). **No verdict changes.**

Addition 4b headline (NEW multi-core compute, the "flip") — the decisive check:

approach	buggy MAPE	corrected MAPE	Δ
naive-linear	23.03%	23.03%	0.00
analytical roofline	**21.60%**	**21.60%**	0.00
bottleneck-gated	**22.30%**	**22.30%**	0.00
learned quantile (p50)	24.00%	23.38%	−0.62pp

Significance: naive-linear vs. analytical CI **[0.30, 2.55]**, identical to the buggy-data result — significant, same direction, same magnitude. naive-linear vs. gated CI **[0.28, 1.27]**, also identical. **The multi-core flip survives on corrected data, to the same two decimal places.** Mechanism: the gate/weight both formulas actually use, `p_compute`, is what matters, and it doesn't move for the queries this headline depends on — q1/q12/q16 (the compute-labeled queries) stay at `p_compute` ≈ 0.99–1.0 in both the buggy-trained and corrected-trained classifier. The reclassification the correction does cause (q17, q18, and others swapping among bandwidth/io/mixed) redistributes weight only between the `p_bandwidth` + `p_io` and `p_mixed` terms, never touching the compute-vs-not decision this result rests on.

Verdict

- **Inflation is not constant: 3.06x–12.00x across the 21-query workload**, driven by plan node count. Confirmed by direct local regeneration, not inferred.
- **The bottleneck classifier changed materially, and got worse, not just different: macro F1 0.562 → 0.375, io-class recall 0.25 → 0.00.** This is now understood mechanistically (correction removes the only cross-machine feature variation the classifier had) and should be reported as the real number going forward, not the original 0.562.
- **Every headline scaling result survives unchanged**: naive/naive-linear MAPE identical (mechanically buffer-free), analytical/gated MAPE identical to 2 decimals under both the old single-core and new multi-core compute signals, and every statistical-significance verdict — including the Addition 4b multi-core-beats-naive-linear flip — is preserved with near-identical bootstrap CIs. The learned GBR model shifts by ≤1.5pp, well inside its own multi-point confidence interval, changing no conclusion.
- **Recommended limitations-section sentence**: "`tpch_dataset.csv`*'s buffer columns (*`shared_hit`*,* `shared_read`*, and the derived* `total_buffers` / `io_ratio`*) were inflated 4–12x by a since-fixed buffer-summation bug in the collection script; because buffer counts are hardware-independent, correct values were regenerated on a locally-verified matching PostgreSQL SF1 instance and substituted per query (*`tpch_dataset_corrected.csv`*) — this materially weakened the bottleneck classifier (macro F1 0.56 → 0.38, io-class recall to 0) but left every scaling-prediction headline result, including the multi-core compute-signal finding, unchanged to within 2 decimal points of MAPE.*"

Artifacts

- `results/buffer_bug_correction_table.csv` — per-query correct vs. buggy buffer values and inflation factors.
- `tpch_dataset_corrected.csv` — corrected dataset (original `tpch_dataset.csv` untouched, kept for provenance).